🎯 "Terraform Interview Guide for DevOps & Solutions Architects role

Top Terraform Interview Questions with real-world project experience!

Whether you're preparing for DevOps roles, platform engineering, or cloud infra interviews — this guide dives into what really matters in **production**.

---

1. Q: How do you manage infrastructure across multiple environments using Terraform?

A:

- Use **workspaces** (terraform workspace) to manage environment isolation (**dev**, **stage**, **prod**).
- Separate **state files** per environment.
- Use environment-specific **variables** or .tfvars files.
- Organize code into **modules** reused across environments.

Scenario: We managed **dev**, **stage**, and **prod** using **workspaces**. **Modules** were reused with environment-specific **variables**. The terraform workspace command ensured the correct **backend** and **state** were used per environment, avoiding **resource conflicts**.

---

2. Q: How do you structure a Terraform project for large-scale infrastructure?

A:

- Use a **modular architecture**: separate **modules** for **VPC**, **compute**, **RDS**, etc.
- Keep reusable logic in modules/ and environment-specific code in envs/.
- Leverage **remote state** with backends like **S3 + DynamoDB** for locking.
- Follow **naming conventions** and **directory standards**.

Scenario: For a large-scale migration, we used a **mono-repo** with shared **modules**. Teams deployed via terraform apply using **S3** backend and **DynamoDB** for state locking.

---

3. Q: How do you manage Terraform state securely and reliably?

A:

- Store state remotely using a **backend** (e.g., **S3**, **Azure Blob**).
- Enable **state locking** with **DynamoDB**.
- Encrypt state at rest using **KMS**.
- Restrict access via **IAM**; avoid committing state to **Git**.

Scenario: State stored in **S3**, encrypted with **KMS**, locked using **DynamoDB**. Access controlled via **IAM policies** to prevent concurrent runs.

---

4. Q: What is Terraform's depends_on and when would you use it?

A:

- Forces an **explicit dependency** between resources.
- Use when **implicit dependencies** are not detected (e.g., null_resource, **provisioners**).
- Prevents **race conditions** during terraform apply.

Scenario: aws_instance depended on a null_resource for setup script. We used depends_on = [null_resource.setup] to control **execution order**.

---

5. Q: How do you implement Blue/Green deployments with Terraform?

A:

- Use separate **modules** or configs for **blue** and **green** stacks.
- Switch traffic via **ALB Target Group** or **Route 53**.
- Use terraform workspace or parallel stacks for management.
- Decommission old stack after **validation**.

Scenario: Deployed v1 and v2 ECS using separate **modules**. **Route 53** updated to **green**, and terraform destroyed old version post validation.

---

6. Q: How do you manage secrets in Terraform without exposing them in code?

A:

- Avoid hardcoding secrets in .tf or .tfvars.
- Use **environment variables** (TF_VAR_secret_name).
- Use **SOPS**, **Vault**, or **AWS SSM** with data sources.
- Inject via **CI/CD secrets management** during runtime.

Scenario:
Used **AWS Secrets Manager** + data "aws_secretsmanager_secret_version" to inject secrets securely into **RDS**, avoiding exposure in code or state.

---

7. Q: How do you handle resource drift in Terraform?

A:

- Use `terraform plan` regularly.
- Enable **drift detection** in CI/CD.
- Run `terraform refresh` to sync state.
- Use `terraform import` for external resources.

Scenario:
A pipeline ran `terraform plan -detailed-exitcode`, detected drift in **EC2 tags**, and triggered a controlled `terraform apply`.

---

8. Q: What is the role of `terraform import` and when do you use it?

A:

- Brings existing infra under Terraform management.
- Requires manual config first.
- One-time use; does **not** modify resources.

Scenario:
Imported 20+ manually created **S3 buckets** using `terraform import aws_s3_bucket.example my-bucket` after writing Terraform code.

---

9. Q: How do you manage versioning of Terraform modules and providers?

A:

- Lock providers in `required_providers` block.
- Pin **module versions** via Git tags or **Terraform Registry**.
- Use **version constraints**: `>=`, `~>`, or exact.
- Use `terraform.lock.hcl` for consistency.

Scenario:
All **providers** and **GitHub modules** were pinned. `terraform init` enforced consistent `terraform.lock.hcl` across teams.

---

10. Q: How do you validate and lint Terraform code?

A:

- Use `terraform validate` for syntax checks.
- Use **tflint** or **checkov** for static analysis.

- Format with terraform fmt.
- Integrate checks in **CI pipelines**.

Scenario:
GitLab CI integrated tflint, terraform validate, and **Checkov** to block unformatted or insecure PRs.

---

11. Q: What is a remote backend and why use it?

A:

- Stores state in a **centralized** and **versioned** location.
- Enables **team collaboration** and **state locking**.
- Backends: **S3**, **Azure Blob**, **GCS**, **Terraform Cloud**.

Scenario:
Used **S3 + DynamoDB**, with **IAM** roles per environment. Enabled secure and auditable remote state management.

---

12. Q: How do you handle cross-account or cross-region resources in Terraform?

A:

- Use multiple **provider blocks** with alias.
- Configure **assume_role** in **AWS provider**.
- Reference data via **outputs** or terraform_remote_state.

Scenario:
Used aliased **providers** and **cross-account roles** to peer **VPCs** across multiple **AWS accounts**.

---

13. Q: What are Terraform modules and how do you use them?

A:

- Reusable blocks for infra components.
- Use source, variables, outputs, locals.
- Break complex infra into isolated units.

Scenario:
Created reusable **modules** for **VPC**, **EC2**, **RDS**, consumed by **dev** and **prod** with different inputs.

---

14. Q: What is local-exec and remote-exec provisioner? Should you use them?

A:

- local-exec: runs locally where Terraform runs.
- remote-exec: runs via **SSH** on provisioned resources.
- Avoid for idempotent infra. Prefer **Ansible** or config tools.

Scenario:
Initially used remote-exec to bootstrap **EC2**. Later replaced with **Ansible** in pipeline for better consistency.

---

15. Q: How do you manage Terraform in a CI/CD pipeline?

A:

- Use stages: terraform init → plan → apply.
- Separate plan and apply with **approval gates**.
- Store **backend credentials** in secret managers.
- Run terraform fmt, validate, and tflint in PRs.

Scenario:
In **GitHub Actions**, terraform plan posted as comments. terraform apply triggered after manual approval. **AWS credentials** injected securely from **GitHub Secrets**.

---

16. Q: How do you manage dependencies between resources in Terraform?

A:

- Terraform uses **implicit dependencies** via references.
- Use depends_on for **explicit dependencies** when references are indirect.
- Avoid **circular dependencies** using modular design.

Scenario:
Used depends_on = [aws_db_parameter_group.pg] to control provisioning order for **RDS** relying on a **parameter group**.

---

17. Q: What is the use of count and for_each in Terraform?

A:

- count: simple replication, index-based.
- for_each: iterates over **maps** or **sets**, better for naming and structure.
- Use for_each for more control, especially with **non-identical** resources.

Scenario:
Provisioned **subnets** using for_each on a map of names and **CIDRs** for better readability vs. count.

---

18. Q: What is terraform taint and when would you use it?

A:

- Marks a resource for **forced recreation**.
- Used when a resource is **unhealthy**, **corrupted**, or misbehaving.

Scenario:
Used terraform taint aws_instance.app to recreate a faulty **EC2 instance** without code change.

---

19. Q: How do you enforce policies or compliance in Terraform code?

A:

- Use **Sentinel**, **OPA**, **Checkov**, **tfsec**, or **terrascan**.
- Block **insecure configurations** (e.g., open **SGs**, unencrypted volumes).
- Integrate static analysis into **CI/CD pipelines**.

Scenario:
Blocked PRs introducing unencrypted **S3 buckets** using **Checkov**, enforcing **encryption compliance**.

---

20. Q: What is the purpose of terraform output?

A:

- Exposes **values** (e.g., URLs, IPs) from state.
- Useful for **CI/CD**, cross-module, or cross-stack references.
- Use sensitive = true to hide output values.

Scenario:
Used terraform output app_url to update **DNS records** and post **Slack notifications** after apply.

---

21. Q: What's the difference between locals and variables in Terraform?

A:

- variables: accept **external input**.
- locals: define **internal expressions** or constants.
- locals are not overridable externally.

Scenario:
Used locals for **tag logic** and variables for environment-specific values like **region**.

---

22. Q: What's the benefit of using terraform plan -out=plan.tfplan?

A:

- Saves a **binary plan** for review and reproducibility.
- Ensures the **reviewed plan** is exactly what gets applied.
- Useful for **approval workflows** in pipelines.

Scenario:
Stored plan.tfplan as an artifact in **CI** and applied only after **security review**.

---

23. Q: How do you manage large Terraform codebases across teams?

A:

- Use **modules** and separate **folders** or **repos**.
- Use **remote state outputs** for cross-team dependencies.
- Apply consistent **naming conventions** and documentation.

Scenario:
Shared **network module** provided **VPC outputs** to app teams via terraform_remote_state.

---

24. Q: What are data sources in Terraform and how are they used?

A:

- Fetch details from **existing infrastructure**.
- Reference AMIs, **VPCs**, secrets, etc.
- Used with **interpolation** or output for dynamic config.

Scenario:
Used `data "aws_ami"` to dynamically select **Amazon Linux 2 AMIs** for updates.

---

25. Q: How do you migrate Terraform state?

A:

- Use `terraform state mv` to move resources.
- Use `terraform init -migrate-state` when changing **backends**.
- Use `terraform state pull/push` for **backup/restore**.

Scenario:
Migrated from local to **S3 backend** using `terraform init -backend-config=...` with **DynamoDB locking**.

---

26. Q: How do you implement resource lifecycle policies in Terraform?

A:

- Use the `lifecycle` block:
  - `prevent_destroy` to protect
  - `create_before_destroy` for zero downtime
  - `ignore_changes` for specific attributes

Scenario:
Used `prevent_destroy = true` on **RDS** and `create_before_destroy = true` for smooth upgrades.

---

27. Q: What is a Terraform provider and how do you manage multiple providers?

A:

- **Providers** interact with APIs (e.g., **AWS**, **Azure**, **GitHub**).
- Use **aliases** for multiple provider configs.
- Supports **multi-region** or **multi-account** workflows.

Scenario:
Used `provider "aws"` and `provider "github"` in the same plan to provision **IAM roles** and **GitHub repos**.

---

28. Q: What's the difference between `terraform destroy` and `terraform apply -destroy`?

A:

- terraform destroy: interactive deletion.
- terraform apply -destroy: non-interactive using saved plan.

Scenario:
In automation, used terraform plan -destroy -out=tfdestroy.plan followed by terraform apply tfdestroy.plan.

---

29. Q: How do you reference outputs from one Terraform state in another?

A:

- Use terraform_remote_state **data source**.
- Access like data.terraform_remote_state.vpc.outputs.vpc_id.

Scenario:
Consumed **VPC outputs** from a shared networking stack using terraform_remote_state.

---

30. Q: How do you manage rollbacks in Terraform?

A:

- Terraform has **no native rollback**.
- Use **version control** to revert config.
- Keep **state snapshots** in backend (**S3 versioning**).

Scenario:
Reverted a Git commit that caused bad **SG config**. Restored old state from **S3 version history**.

---

31. Q: How do you share infrastructure outputs between Terraform modules?

A:

- Use output blocks to expose values.
- In parent module, access with module.<name>.output_name.
- For cross-project, use terraform_remote_state.

Scenario:
ECS module consumed **subnet IDs** from **VPC module** outputs like module.vpc.private_subnet_ids.

---

32. Q: How do you handle multi-cloud infrastructure using Terraform?

A:

- Use different **providers** (aws, azurerm, google, etc.).
- Alias each **provider** if needed.
- Use **modules** or **workspaces** to isolate cloud logic.

Scenario:
Frontend deployed to **Azure**, backend to **AWS** using aliased **providers** in separate **modules**.

---

33. Q: What's the difference between terraform apply and terraform plan?

A:

- terraform plan: shows potential changes.
- terraform apply: applies actual changes and updates state.
- Use plan for review, apply after approval.

Scenario:
CI used terraform plan on **merge requests**, followed by terraform apply after team sign-off.

---

34. Q: How do you secure Terraform backend credentials in CI/CD?

A:

- Store credentials in **secret managers** (e.g., GitHub Secrets).
- Use **OIDC** or **IAM roles** for temporary access.
- Never store credentials in code or state.

Scenario:
OIDC-based IAM role assumption was used in **GitHub Actions** to access AWS securely without hardcoding secrets.

---

35. Q: What is a Terraform workspace, and when should you use it?

A:

- **Workspaces** isolate **state files** under a shared configuration.
- Ideal for **dev/stage/prod** environments.

- Use directories/modules for complex infra instead.

Scenario:
Used `terraform workspace` for isolated state while sharing codebase across **dev**, **stage**, and **prod**.

---

36. Q: How do you define reusable and configurable Terraform modules?

A:

- Structure with `main.tf`, `variables.tf`, `outputs.tf`.
- Accept inputs via `variable` blocks.
- Output important values using `output` blocks.

Scenario:
The `ec2_instance` module accepted **AMI**, **tags**, **instance type**, and **user data**; reused across projects with Git tagging.

---

37. Q: What is the purpose of `terraform init`?

A:

- Initializes working directory.
- Downloads **providers** and **modules**.
- Sets up **backends** and lock files.

Scenario:
Ran `terraform init -migrate-state` to shift from **local backend** to **S3** during a migration.

---

38. Q: How can you enforce resource tagging in Terraform?

A:

- Define standard tags via `locals`.
- Merge with resource-level or custom tags.
- Enforce via **Checkov**, **OPA**, or **AWS Config Rules**.

Scenario:
Used `locals { common_tags = { Environment = var.env } }` and merged into every `tags` block.

---

39. Q: How do you test Terraform code before deploying to production?

A:

- Use terraform plan, terratest, or kitchen-terraform.
- Run in **staging** or **sandbox** workspaces.
- Automate with **CI/CD**.

Scenario:
Used **Terratest** to validate infra, then destroy it post-tests for **EC2**, **ALB**, and **RDS** connectivity checks.

---

40. Q: What are the benefits of Terraform Cloud or Enterprise?

A:

- Hosted **remote backend** with collaboration.
- **Policy enforcement** with **Sentinel**.
- VCS integration, **RBAC**, cost estimates, **UI workflow**.

Scenario:
Used **Terraform Cloud** with **GitHub integration**. PRs triggered plans, and **Sentinel** policies blocked untagged or insecure resources.

---

41. Q: How do you manage blue/green infrastructure deployment using modules?

A:

- Use two **modules** (e.g., module.ecs_blue, module.ecs_green).
- Switch traffic using **ALB** or **Route 53**.
- Destroy old stack post-validation.

Scenario:
Deployed two **ECS services**, shifted **target groups**, and destroyed the old service via separate plan.

---

42. Q: How do you handle conditional logic in Terraform?

A:

- Use **ternary operators** (condition ? A : B).
- Use count or for_each with conditionals.

- Use **dynamic blocks** for nested conditions.

Scenario:
Created **CloudWatch alarms** only in **prod** using `count = var.env == "prod" ? 1 : 0`.

---

43. Q: How do you manage complex Terraform variable types?

A:

- Use `map`, `list`, or `object` with `type` constraints.
- Transform via `locals`.
- Use `jsondecode()` in CI/CD for structured inputs.

Scenario:
Used `object` variable for defining IAM roles with inline policies. Parsed and validated structure dynamically.

---

44. Q: How do you ensure backward compatibility in shared Terraform modules?

A:

- Avoid breaking inputs/outputs.
- Use `default` values and **semantic versioning**.
- Add **deprecation warnings** in comments or logs.

Scenario:
Added optional variables with defaults. Older versions didn't break. Used Git tags to manage rollout.

---

45. Q: What are common mistakes to avoid in Terraform?

A:

- Committing `.tfstate` files to Git.
- Hardcoding secrets or **values**.
- Not locking **provider versions**.
- Ignoring **drift** or `terraform plan`.
- Using `count` with changing indexes.

Scenario:
Switched from `count` to `for_each` after instance **replacement issues** due to index changes.

---

46. Q: How do you implement Terraform in a secure CI/CD pipeline?

A:

- Store **AWS credentials**, **backend config** in **secret managers**.
- Use **OIDC federation** and **least-privilege IAM roles**.
- Split plan/apply with **approval stages**.
- Add terraform validate, tfsec, **checkov** scans.

Scenario:
OIDC-based IAM role with **GitLab pipeline**, only **admins** could run terraform apply with **MFA**.

---

47. Q: How do you use Terraform for cross-region networking (e.g., VPC Peering)?

A:

- Configure **multiple provider aliases** per region.
- Use provider = aws.<alias> in resources.
- Share **outputs** across modules.
- Manage **route tables**, **transit gateways** carefully.

Scenario:
Setup **VPC peering** between **Ohio** and **N. Virginia**, using provider "aws" { alias = "us_east_2" }.

---

48. Q: What is terraform state rm and when would you use it?

A:

- Removes a resource from Terraform **state** without destroying it.
- Useful for **manual management**, **resource transfer**, or fixing **state drift**.
- Use carefully — can cause **inconsistencies**.

Scenario:
Ran terraform state rm for a **CloudWatch alarm** handed off to the **monitoring team**.

---

49. Q: How do you manage concurrency in Terraform for large teams?

A:

- Use **remote state** with **state locking** (e.g., S3 + DynamoDB).

- **Serialize applies** in CI/CD using **queues** or **locks**.
- Avoid terraform apply on dev machines.

Scenario:
Race conditions were fixed by enforcing apply only via **GitHub Actions** + **DynamoDB locking**.

---

50. Q: How do you secure Terraform state files?

A:

- Enable **encryption at rest** (e.g., S3 + KMS).
- Turn on **S3 versioning** and **access logging**.
- Use **IAM role-based access control**.
- Avoid local .tfstate in shared environments.

Scenario:
Each environment had its own **IAM role**, **KMS key**, and blocked **S3 public ACLs**.

---

51. Q: How do you implement feature toggles (on/off flags) in Terraform?

A:

- Use variables: count = var.enable_feature ? 1 : 0.
- Combine with locals, dynamic blocks.
- Use in **multi-module systems**.

Scenario:
Controlled **RDS enhanced monitoring** using var.enable_enhanced_monitoring to skip it in **staging**.

---

52. Q: How do you manage Terraform provider version upgrades safely?

A:

- Use **required_providers** block to pin versions.
- Review **changelogs**, test in **isolated branches**.
- Use terraform lock file for consistency.

Scenario:
While upgrading from **AWS provider v4.50 → v5.0**, deprecated fields were removed and patched in **staging first**.

---

53. Q: How do you protect production infrastructure from accidental changes?

A:

- Use prevent_destroy lifecycle rules.
- Add **approval gates** in CI/CD.
- Enforce **Sentinel/OPA policies**.
- Apply only with **privileged IAM roles**.

Scenario:
RDS, **S3** had prevent_destroy = true, and only admins with **MFA** could run applies.

---

54. Q: How do you manage large, multi-module Terraform projects?

A:

- Organize by modules/, envs/, shared/.
- Use **input variable naming conventions**.
- Use **Terragrunt** or per-module apply.
- Leverage **terraform_remote_state** for sharing outputs.

Scenario:
12+ modules (VPC, compute, logging) ran independently, each with isolated **backend config**.

---

55. Q: What's the difference between for_each and count, and when to use each?

A:

- count: index-based, for identical resources.
- for_each: map/set based, stable identifiers.
- for_each avoids **resource recreation** on list change.

Scenario:
Switched to for_each after **IAM user renaming** with count triggered unintended re-creations.

---

56. Q: How do you roll back Terraform changes after a failed apply?

A:

- Revert the **config** via version control.
- Use terraform plan to verify changes.
- Restore state using **S3 versioning** or **backups**.

Scenario:
Wrong **subnet CIDR** was fixed by rolling back the Git commit and restoring old `.tfstate` version.

---

57. Q: How do you automate cost-saving practices with Terraform?

A:

- Use **EventBridge**, **cron**, or **Lambda** to schedule instance shutdowns.
- Enable **S3 lifecycle policies**, use **spot instances**.
- Monitor using **tags** and **budgets**.

Scenario:
Set EC2 shutdown at 8 PM via **Lambda**, and archived **S3 logs** after 30 days using **lifecycle policies**.

---

58. Q: How do you implement multi-tenancy with Terraform?

A:

- Use `for_each` with tenant maps.
- Isolate using **modules**, **workspaces**, or **directories**.
- Store metadata in external **YAML/JSON**.

Scenario:
Used `for_each` to create **S3 buckets**, **IAM roles**, **EKS namespaces** for each **tenant** with custom tags.

---

59. Q: How do you simulate infrastructure changes before deploying?

A:

- Use `terraform plan` and store with `-out=plan.tfplan`.
- Run in **sandbox/staging** first.
- Review via **CI pipeline logs**.

Scenario:
PRs triggered `terraform plan`, and changes were reviewed by **security** before apply.

---

60. Q: How do you integrate Terraform with GitHub Actions or GitLab CI/CD?

A:

- Define jobs: init → validate → plan → apply.
- Authenticate via **OIDC** or **secrets**.
- Use **job artifacts** to persist plans.

Scenario:
GitHub Actions used **OIDC** to assume **AWS role**, then ran terraform plan. Manual approval triggered terraform apply.

61. Q: How do you implement DR (Disaster Recovery) strategies using Terraform?

A:

- Use region-agnostic **modules**.
- Duplicate critical infra in **secondary regions**.
- Use **S3 Cross-Region Replication**, **RDS Read Replicas**, **DynamoDB Global Tables**.
- Use terraform workspace or **DR-specific variables**.

Scenario:
Built warm-standby in us-east-1 and eu-west-1. Used **Aurora Global DB**, replicated **S3**, and failover via **Route 53**.

---

62. Q: How do you enforce compliance or security rules in Terraform configurations?

A:

- Use tools like **tfsec**, **checkov**, **OPA**, **Sentinel**.
- Block insecure patterns: open **SGs**, unencrypted **volumes**, missing **tags**.
- Use **pre-commit hooks** and CI/CD scanners.

Scenario:
Checkov blocked PRs with untagged or public resources. **GitHub Actions** enforced security gates before apply.

---

63. Q: How do you enable collaboration between multiple teams on the same Terraform project?

A:

- Isolate **remote state** per module or environment.
- Define **clear module ownership boundaries**.
- Use **Terragrunt** or **Terraform Cloud** for orchestration.
- Standardize **naming/tagging** across teams.

Scenario:
Networking team managed VPC. App teams used terraform_remote_state to consume **VPC outputs** securely.

---

64. Q: How do you manage lifecycle events like create_before_destroy for zero-downtime?

A:

- Use lifecycle block with create_before_destroy = true.
- Avoid using count when resource **order matters**.
- Combine with depends_on if needed.

Scenario:
Enabled create_before_destroy on **RDS** during major upgrade to avoid database downtime.

---

65. Q: How do you manage module versioning and upgrades in Terraform?

A:

- Use **Git tags** or **Terraform Registry** versions (~> 1.2).
- Test in **non-prod** before promoting.
- Maintain **changelog** and **deprecation logs**.

Scenario:
EKS module versioned with ?ref=vX.X.X. Upgrades followed **semver**, with rollback path and **migration docs**.

---

66. Q: How do you track who made what changes in Terraform?

A:

- Use **Git history**, **PR approvals**.
- Enable **S3 access logs**, **DynamoDB streams**, or **Terraform Cloud audit logging**.
- Log Git metadata in **CI apply jobs**.

Scenario:
Each apply recorded Git SHA, branch, and user via **env vars**. **S3 versioning** kept .tfstate history.

---

67. Q: How do you handle large Terraform state files?

A:

- Split infra into **smaller modules** and **separate state files**.
- Avoid managing **heavy data** (e.g., secrets) via Terraform.
- Use terraform_remote_state for inter-module access.

Scenario:
Monolithic state (>1MB) was split into 6 isolated modules: **network**, **compute**, **DB**, **monitoring**, etc.

---

68. Q: How do you make Terraform configurations portable across regions or accounts?

A:

- Parameterize using **variables**: region, account_id, AZs.
- Use data "aws_caller_identity" and data "aws_availability_zones" to avoid hardcoded values.

Scenario:
Used **lookup tables** and **data sources** to deploy same module across multiple AWS accounts.

---

69. Q: How do you avoid unintended resource deletions in Terraform?

A:

- Use prevent_destroy.
- Add **approval stages**, plan -detailed-exitcode in CI.
- Review terraform plan before apply.

Scenario:
Pipeline blocked terraform apply after exit code 2 indicated deletions. Root cause: misconfigured variable.

---

70. Q: How do you deploy Terraform across multiple environments (dev, stage, prod)?

A:

- Use dedicated folders, pipelines, or **workspaces**.
- Maintain separate **backend configs**.
- Use .tfvars or env-specific **variables**.

Scenario:
envs/dev/, envs/stage/, envs/prod/ used individual backends. **Jenkins** injected TF_VAR_env.

---

71. Q: How do you improve performance for large Terraform plans?

A:

- Split into **modules**.
- Avoid excessive **data sources**, simplify logic.
- Cache **providers/plugins** in CI.
- Use `apply -target` for changed modules.

Scenario:
Cut plan time from 5 min to 90 sec by modularizing infra and using targeted applies like `-target=module.eks`.

---

72. Q: How do you automate documentation for Terraform modules?

A:

- Use **terraform-docs**, **tfplugindocs**.
- Auto-generate `README.md` with inputs/outputs.
- Run via **pre-commit hooks** or CI.

Scenario:
Each module had `terraform-docs . > README.md` on PR. Docs stayed up-to-date without manual effort.

---

73. Q: How do you dynamically generate resources in Terraform?

A:

- Use `for_each` or `count` with maps/lists.
- Combine with `locals` and **conditionals**.
- Use `dynamic` blocks for nested configs.

Scenario:
Generated **CloudWatch alarms** using `for_each = var.alarm_map` with environment-specific thresholds.

---

74. Q: How do you handle conditional nested blocks (like SG rules) in Terraform?

A:

- Use dynamic blocks inside resources.
- Control creation using for_each or conditionals.
- Refactor into **submodules** for complex logic.

Scenario:
Created dynamic ingress rules in aws_security_group only when ports were defined in input variables.

---

75. Q: How do you ensure Terraform modules are reusable, maintainable, and scalable?

A:

- Use structured files: main.tf, variables.tf, outputs.tf.
- Follow **semantic versioning**, **changelogs**, and **test coverage** (e.g., **Terratest**).
- Keep logic simple, focused, and documented.

Scenario:
Internal module registry followed checklist: **README**, terraform-docs, **pre-commit**, **versioned tags**, **Terratest**.